Faculty of Media Engineering and Technology
German University in Cairo
Dr. Nourhan Ehab

CSEN 903: Advanced Computer Lab, Winter 2026
Lab 3 Manual: Basics of Neural Networks with Tensorflow

Part 1: Read and Watch Before the Lab

In the previous lab, we explored **supervised learning**, where models learn from labeled data to make predictions on new, unseen inputs. We used a traditional machine learning model for training and prediction. In this lab, our focus shifts to **neural networks**, moving beyond classic machine learning approaches.

Neural networks are powerful machine learning models inspired by the structure of the brain. In this lab, we introduce **Feedforward Neural Networks** using **TensorFlow** and **Keras**. You will learn how to:

1. Define model architecture (input, hidden, and output layers)
2. Choose activation functions (ReLU, sigmoid, softmax)
3. Understand how a network learns (loss, backpropagation, optimizers)
4. Choose a loss function and optimizer
5. Train the model and monitor training progress
6. Evaluate performance on unseen data

We will continue using the **Diamonds Dataset**, treating the cut prediction task as a multi-class classification problem.

Required Readings:

1. TensorFlow Keras Guide: Getting Started
2. TensorFlow Tutorial: Basic Classification
3. Introduction to Neural Networks
4. Activation Functions Explained

Required Watch List:

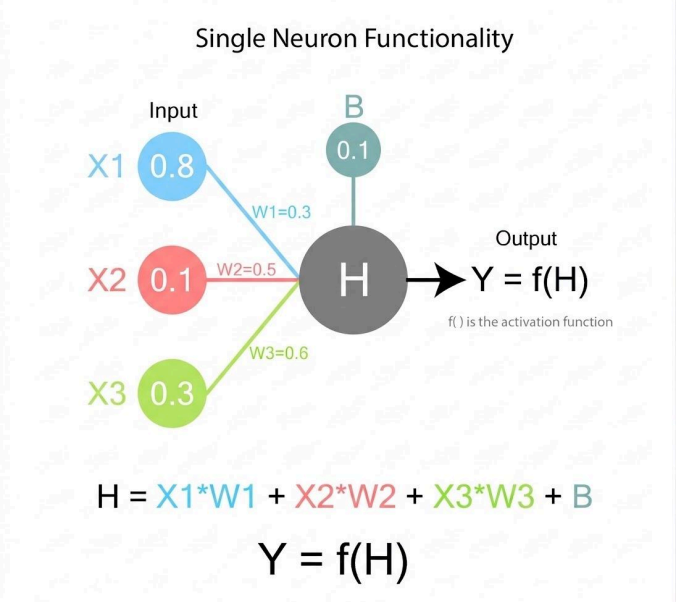
1. Neural Networks and Activation Functions
2. [Extra Material] Backpropagation

Faculty of Media Engineering and Technology
German University in Cairo
Dr. Nourhan Ehab

Neural Network Model:

Neural network models are built from layers, and each layer is made up of neurons. Each neuron is influenced by the outputs of neurons from the previous layer, while neurons within the same layer are not directly connected. A neuron itself is a simple function. To see how this works, let's look at a real-life example using just one neuron.

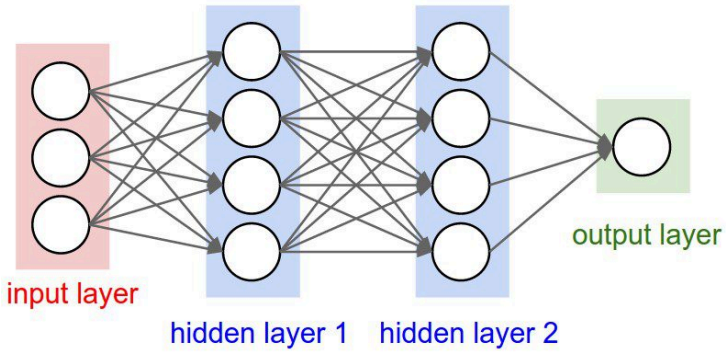
In this simple example, the neuron is deciding whether a person will go to the cinema or not based on three main factors: **Weather (X1)**, **Company (X2)**, and **Proximity (X3)**. Each factor, however, does not contribute equally to the decision. Some may be more important than others. To reflect this, each input is multiplied by a weight (**W1, W2, W3**) that adjusts its influence. A bias term (**B**) is also added to give flexibility to the decision. All of these are combined into a single value **H**, calculated as:



This value is then passed through a function **f(H)**, which outputs a clear decision: **Yes** (go to the cinema) or **No** (don't go). In this way, the neuron is essentially weighing the importance of weather, company, and proximity to decide the final outcome.

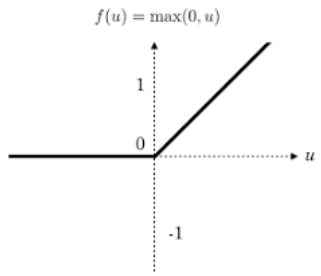
Function f(H) is referred to as an **activation function.** Activation functions introduce non-linearity, which is crucial because it allows the model to move beyond simple linear relationships and learn complex patterns in the data. More details about activation functions are to follow.

A single neuron makes simple decisions, but stacking neurons into **layers** allows the model to learn more complex patterns and make smarter decisions. The layers in a neural network are generally categorized into three types: the **input layer**, one or more **hidden layers**, and the **output layer**. When a network contains more than two hidden layers, it is commonly referred to as a **deep neural network**.

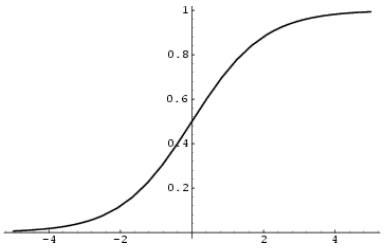


- **Input Layer:** the first layer of neurons in the model, which consists of *n* neurons, where *n* is equal to the number of features (i.e., the dimensionality) of the input data. In other words, the input layer has one neuron per feature in your dataset.
- **Hidden Layer(s):** middle layer(s) in a neural network where inputs are transformed into more useful features for learning.
- **Output Layer:** contains the neurons that produce the model's final predictions. Its size depends on the task: one neuron for binary classification, one per class for multi-class classification, or a single neuron with linear activation for regression.
- **Activation Function:** is like a switch that turns a neuron's input into an output. It lets neural networks learn complex patterns instead of just simple, straight-line relationships. Without activation functions, every neuron would only compute a weighted sum. Stacking layers of weighted sums still gives a weighted sum, so a network with ten layers would be no more powerful than a single linear model. Activation functions add non-linearity, which lets the network learn curved, complex patterns. Below, we will discuss the most commonly used activation functions:
    - **ReLU:** outputs the input if it's positive, otherwise sets it to 0, producing outputs in the range of [0, ∞[. It helps models learn

faster, and works well when you just want to pass along positive values; thus, it is commonly used in hidden layers.



- **Sigmoid:** converts the input values into an S-shaped curve, producing outputs in the range [0, 1], which makes it mainly used for **binary classification problems**.



- **Softmax:** converts a vector of values into probabilities that sum to 1, with each output in the range [0, 1], which makes it suitable for **multi-class classification problems**, when you need to assign probabilities across several categories.

How Does a Network Learn?

1. **Forward pass:** A batch of inputs flows through the layers, and each neuron computes its weighted sum and activation. The output layer produces a prediction, e.g., 0.8 probability the diamond is ideal cut.
2. **Compute the loss:** The loss function compares the prediction to the true label and returns a single number: how wrong the model was.
3. **Backpropagation:** The network needs to know how much each weight contributed to the error. Backpropagation answers this by working backwards from the output layer to the input layer, computing the gradient of the loss with respect to every weight. A gradient tells us: "if I increase this weight slightly, does the loss go up or down, and by how much?"

4. **Update the weights:** The optimizer uses the gradients to nudge each weight in the direction that reduces the loss.. The most common type of optimizer is Adam, and it is the standard in most deep learning models. It is based on SGD.
    a. **SGD (Stochastic Gradient Descent):** updates weights after every single batch instead of waiting for the whole dataset. Faster updates, but each one is based on a small, noisy slice of the data.
    b. **Adam (Adaptive Moment Estimation):** builds on SGD by remembering the direction of past updates and adjusting each weight's step size individually. This is usually faster and more stable, with less manual tuning.
5. **Repeat.** This is done for every batch. One full pass over the training data is one epoch, and training repeats for many epochs until the loss stops improving.

You don't need to compute gradients by hand. TensorFlow does this automatically. For the full derivation, see the extra backpropagation video.

Part 2: In-Lab Task In Lab Notebook

1. Set Up Your Environment.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.preprocessing import OrdinalEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping

np.random.seed(42)
tf.random.set_seed(42)
```

2. Load and prepare the data. You can refer to the Lab Manual 1 for general information about Data Cleaning, or Lab Manual 2 for these specific cleaning steps.

```
df =
pd.read_csv('/kaggle/input/datasets/nancyalaswad90/diamonds-prices/Diamonds
Prices2022.csv', index_col=0)
df = df.drop_duplicates()

cut_order = ['Fair', 'Good', 'Very Good', 'Premium', 'Ideal']
color_order = ['J', 'I', 'H', 'G', 'F', 'E', 'D']
clarity_order = ['I1', 'SI2', 'SI1', 'VS2', 'VS1', 'VVS2', 'VVS1', 'IF']

encoder = OrdinalEncoder(categories=[color_order, clarity_order, cut_order])
df[['color', 'clarity', 'cut']] = encoder.fit_transform(df[['color',
'clarity', 'cut']])
df.head()
```

We usually **scale features** before training machine learning models because raw features can have very different ranges and units, which can cause problems for many algorithms. Think about features in our Diamonds dataset. One of the features is **Carat (carats, range ~0.2–5.01),** and another is **Price (dollars, range ~326–18223)**.

Without scaling, algorithms that depend on distances or weights may treat *price* as much more important than *carat*, just because the numbers are bigger. Standardization puts them on the same scale. Standardization means transforming each feature so that it has:

- **Mean = 0**
- **Standard deviation = 1**

```
X = df.drop(columns=['cut'])
y = df['cut'].astype(int)
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.2,
random_state=42, stratify=y )

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print(f"Train: {X_train.shape}, Test: {X_test.shape}")
```

3. Build a Logistic Regression Model:

```
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)

y_pred_lr = log_reg.predict(X_test_scaled)
print("Logistic Regression - Classification Report (Test Set)")
print(classification_report(y_test, y_pred_lr, target_names=cut_order))
```

4. Build 2 Neural Network Models, define first the function:

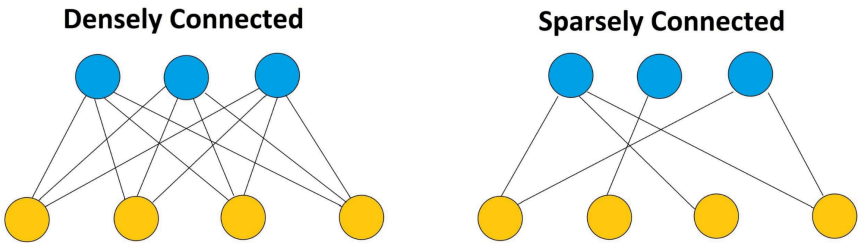
```
def build_model():
    model = keras.Sequential([
    layers.Input(shape=(X_train_scaled.shape[1],)),
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(16, activation='relu'),
    layers.Dense(len(cut_order), activation='softmax')
])
    model.compile( optimizer='adam', loss='sparse_categorical_crossentropy',
    metrics=['accuracy'] )

return model
```

In the above code snippet, **Input(shape=(X_train.shape[1],))** creates the model's input layer with one neuron for each feature in the training data.

The first hidden layer is a fully connected (dense) one with **64 neurons**. It means the layer learns 64 features from the previous layer's (i.e., input layer) outputs. While the activation **ReLU** makes the 64 neurons output either the input value (if positive) or 0 (if negative).

A fully connected network means every neuron in one layer is connected to **all** neurons in the next layer. Each connection has its own weight, allowing the network to learn complex relationships between features.



The second hidden layer is also fully connected (dense) with **32 neurons**, meaning it learns 32 new features from the outputs of the first hidden layer. The **ReLU** activation makes these 32 neurons output either the input value (if positive) or 0 (if negative). After that there is another layer with 16 neurons.

The output layer has 5 neurons (one for each cut category), with a softmax activation function that converts the raw outputs into a probability distribution across all five classes. For example, `0.7` for `Ideal` and smaller values for the rest, all summing to 1. Softmax is used here instead of sigmoid because this is a multi-class problem. We need probabilities spread across five categories, not a single yes/no probability the way sigmoid would give for a binary problem.

After defining the model architecture, the next step is to compile it.

Compiling the model sets up how it will learn by choosing an **optimizer** (how weights update), a **loss function** (what the model tries to minimize), and evaluation **metrics** (Refer to Lab 2 for revisiting other evaluation metrics that can be used). How this works is as follows:

1. The model makes predictions.
2. The loss function measures the error (the smaller the loss value, the better our model is performing). It gives a single number representing "how bad" the model is doing on training data by matching the model's predictions against the actual ground truths. We will discuss common loss functions and when to use each:
    a. **Binary Cross-Entropy:**
        i. Used for binary classification tasks (yes/no, spam/not spam, disease/no disease).
        ii. If the true label is `1` (positive) and the model predicts `0.9`, the loss will be small. If it predicts `0.1`, the loss will be large.
    b. **Categorical Cross-Entropy:**

i. Used for multi-class classification (classifying images of cats, dogs, and birds).
ii. It compares the predicted probability distribution with the true class (one-hot encoded).

c. **Mean Squared Error (MSE), Mean Absolute Error (MAE):**
i. Used for regression problems (predicting continuous values, like house prices, temperature).

3. The optimizer uses this error to update the weights in the right direction. An optimizer is the algorithm that updates the weights using the gradients from backpropagation. The simplest is **gradient descent**: move each weight a small step against its gradient. The size of that step is the **learning rate**. Too large and the loss bounces around or diverges. Too small and training is very slow.

We will create a neural network where we train normally and the other with early stopping enabled.

```
neural_model = build_model()
neural_model_with_es = build_model()
```

4. Train the Neural Network Models

```
history_nm = neural_model.fit(
       X_train_scaled,
       y_train,
       validation_split=0.3,
       epochs=200,
       batch_size=32,
       verbose=1 )
```

- A **validation_split** divides your training data into two parts. Earlier, the dataset was split into training and testing sets. Here, we further split the training data into training and validation sets, where the training set is used to train the model, and the validation set helps monitor how well it is learning. The testing set remains separate for final evaluation.
- We use **validation** during training to make sure the model isn't just memorizing the training data but can also perform well on new, unseen data. We will split our data into 70% for training and 30% for validation.

- An **epoch** is one full pass through the entire training dataset, where the model sees and learns from every example once. While in theory, more epochs could improve learning, in practice, there are limitations. Training for too many epochs can be computationally expensive, especially with large datasets or data types like images that require significant processing power. Moreover, using an excessively large number of epochs can lead to overfitting, where the model learns the training data too well and fails to generalize to new, unseen data.
- **verbose=1** means it will show a progress bar with details of the training process.

To avoid overfitting when training with many epochs, we use **early stopping**, which automatically stops training once the validation performance stops improving.

In the code below, we added **early stopping** to prevent overfitting when training the model. It monitors the **validation accuracy** and stops training if it doesn't improve for 15 consecutive epochs (`patience=15`). The parameter `restore_best_weights=True` ensures the model keeps the weights from the epoch with the highest validation accuracy. This way, we avoid wasting time on unnecessary epochs and keep the best-performing version of the model.

```
from tensorflow.keras.callbacks import EarlyStopping
early_stop = EarlyStopping(monitor='val_accuracy', patience=15,
restore_best_weights=True)

history_nm_es = neural_model_with_es.fit(
       X_train_scaled,
       y_train,
       validation_split=0.3,
       epochs=200,
       batch_size=32,
       verbose=1,
       callbacks=[early_stop] )
```

Rather than feeding the entire dataset through the network in a single step, the training data is divided into smaller subsets called **batches**. The model processes one batch at a time and updates its weights after each batch. When all

batches have been processed, the model has completed one full pass through the dataset, which is known as an **epoch**. For example, with `batch_size=32`, the model takes **32 samples**, runs them through the network, calculates the error, and updates weights. Then it takes the next 32 samples, and so on, until it covers the whole dataset (that's one epoch).

5. Visualize the training progress.

```
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Accuracy - overfit model
axes[0, 0].plot(history_nm.history['accuracy'], label='Train')
axes[0, 0].plot(history_nm.history['val_accuracy'], label='Validation')
axes[0, 0].set_title('Model 1 (No Early Stopping) – Accuracy')
axes[0, 0].set_xlabel('Epoch')
axes[0, 0].set_ylabel('Accuracy')
axes[0, 0].legend()

# Loss - overfit model
axes[0, 1].plot(history_nm.history['loss'], label='Train')
axes[0, 1].plot(history_nm.history['val_loss'], label='Validation')
axes[0, 1].set_title('Model 1 (No Early Stopping) – Loss')
axes[0, 1].set_xlabel('Epoch') axes[0, 1].set_ylabel('Loss')
axes[0, 1].legend()

# Accuracy - early stopped model
axes[1, 0].plot(history_nm_es.history['accuracy'], label='Train')
axes[1, 0].plot(history_nm_es.history['val_accuracy'], label='Validation')
axes[1, 0].set_title('Model 2 (Early Stopping) – Accuracy')
axes[1, 0].set_xlabel('Epoch') axes[1, 0].set_ylabel('Accuracy')
axes[1, 0].legend()

# Loss - early stopped model
axes[1, 1].plot(history_nm_es.history['loss'], label='Train')
axes[1, 1].plot(history_nm_es.history['val_loss'], label='Validation')
axes[1, 1].set_title('Model 2 (Early Stopping) – Loss')
axes[1, 1].set_xlabel('Epoch')
axes[1, 1].set_ylabel('Loss')
axes[1, 1].legend()

plt.tight_layout()
plt.show()
```

6. Evaluate on the test set.

```
test_loss_overfit, test_acc_overfit = neural_model.evaluate(X_test_scaled,
y_test, verbose=0)
test_loss_es, test_acc_es = neural_model_with_es.evaluate(X_test_scaled,
y_test, verbose=0)

lr_test_acc = log_reg.score(X_test_scaled, y_test)

comparison = pd.DataFrame({
      'Model': ['Logistic Regression', 'NN – No Early Stopping', 'NN – Early
Stopping'],
      'Test Accuracy': [lr_test_acc, test_acc_overfit, test_acc_es],
      'Test Loss': [np.nan, test_loss_overfit, test_loss_es],
      'Epochs Trained': [np.nan, len(history_nm.history['loss']),
       len(history_nm_es.history['loss'])] })

comparison
```

This line evaluates the trained models on the test data and returns their loss and accuracy, comparing them to each other

Part 3: Graded Task

Task Description:

1. Build two **neural network classifiers with early stopping** using TensorFlow to predict the cut. These neural networks should be as follows:
    a. neural_model_A: Create a neural network with a different structure than the one used above, try to add/move layers and neurons. You can also change activation functions.
    b. neural_model_B: Recreate the exact same early stopping model as we've seen in the guided task but change the optimizer to SGD.
2. Train and evaluate both models.
3. Plot the training/validation accuracy & loss graphs. Compare them with the original neural network with early stopping.
4. Create a table where you compare between your two new models and our

original early stopping model on:

a. Test Accuracy
b. Test Loss
c. Epochs Trained

Deliverables:

Submit an accessible notebook link containing all the completed tasks listed above via the form. The submission **deadline** is by the end of your lab.